



UNIVERSITÀ DI PARMA

Function Pointers

*Memoria est thesaurus omnium rerum et
custos*

De Oratore, Cicerone, I sec a.C.

- Puntatori a funzione
 - Sintassi
 - Definizione
 - Uso
- Funzioni
 - `qsort()`
 - `bsearch()`

SUMMARY



- Abbiamo definito le variabili come area di memoria con nome
 - Quindi hanno un indirizzo
- Ma tutto è in memoria
- Anche il codice
 - Quali porzioni di codice hanno un nome?
 - Le funzioni!

- Come lo ricavo?
- Nome della funzione senza ()
 - Come per array, no operatore &

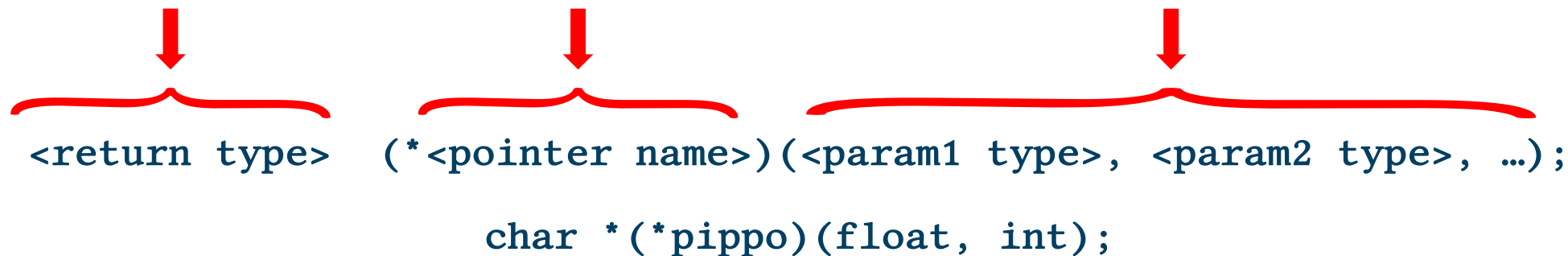
- Per le variabili:
 - Indirizzo
 - Tipo di dato puntato
- Per le funzioni
 - Indirizzo
 - Tipo di dato restituito
 - Elenco e tipi parametri formali

- Sintassi sempre basata su uso *
 - E con ()

Function return type

Pointer name

Function param
types



The diagram illustrates the syntax of a function pointer declaration. It shows the general form: `<return type> (*<pointer name>)(<param1 type>, <param2 type>, ...);`. Red arrows point from the labels 'Function return type', 'Pointer name', and 'Function param types' to their respective parts in the syntax. Red brackets are used to group the parameters in the example.

```
<return type> (*<pointer name>)(<param1 type>, <param2 type>, ...);
```

```
char *(*pippo)(float, int);
```

- Sintassi sempre basata su uso *
 - E con ()
- Esempi di puntatori:
 - `int (*a)(void);` // a funzione che restituisce int e non vuole argomenti
 - `int (*b)(int,int)` // a funzione che restituisce int e prende due int
 - `char (*c)(double*, float*);`

- We can not write or read at that address
 - Forbidden!
 - Therefore, not the same as data pointers
- Main usage
 - Widely used in OS and RTOS programming
 - Vectors of functions (i.e. Interrupt Vector Tables)
 - **Callbacks, namely code that we can forward to other code (typically a function)**
 - We can achieve OOP concepts like inheritance or polymorphism

- A callback is actually a “regular” function
 - Namely, you can invoke it as other functions
- Therefore, “when a function can be considered as a callback”?
- A function is used as a callback when it is passed to another function
 - Namely stored as data in args
 - What we can use to store it? → function pointers!

The term “callback” is also used for asynchronous functions in other languages or contexts (not our case)

Why we need callbacks?

- Sometimes, providing “instructions” on how to deal with data can lead to “smarter” code
- i.e. we want to compute the sum of numbers in an array using a function
 - Some steps are not dependent on the content of the array
 - i.e. iterating on array elements
 - But others are:
 - i.e. adding two numbers is strictly dependent on number types (complex vs natural)
 - We would need as many functions as many kind of data types we want to deal with or...
 - We can use a single function and provide it with the instructions on how to deal with specific types

- C standard library contains two main examples of functions that need a “callback”
 - `qsort()` → arbitrary array sorting
 - `bsearch()` → search in ordered arrays
- They are very good examples of “polymorphism”
 - Namely the capability to operate on different kinds of data

- La funzione `qsort()` permette di ordinare un array generico
 - Quindi array di `int`, `float`, `double`, puntatori, `struct` di differenti tipi...
- Basi di tutti gli algoritmi di ordinamento:
 - Considero gli elementi dell'array a coppie
 - Confronto il loro contenuto
 - Nel caso li scambio di posizione

- Cosa serve per ordinare un array?
 - Per considerare gli elementi dell'array
 - Devo sapere di quanti elementi è composto
 - Passo alla funzione il numero di elementi dell'array
 - Per gli scambi
 - Devo sapere quanti byte occupa ciascun elemento
 - Passo alla funzione la dimensione in byte di un singolo elemento
 - Per il confronto
 - ??

- Se devo ordinare un array generico devo poter confrontare elementi di differenti tipologie
 - Ad esempio, il confronto di due scalari è differente dal confronto tra stringhe
 - Operatore ' $>$ ' vs strcmp()
- Non solo
 - Ordinamento crescente o decrescente
- Soluzione, chi invoca la qsort() deve anche fornire la parte di codice che realizza il confronto

- qsort() takes as inputs:
 - Array to be sorted
 - The number of elements of the array
 - The size of each element
 - The address of a callback for comparing elements

```
void qsort(void *base, size_t nmemb, size_t size,  
int (*compar)(const void *, const void *));
```

- Lets' focus on the callback
 - `int (*compar)(const void *, const void *)`
- It takes as input the address of two elements to be compared
 - As generic void data → internally we need to convert them
- It returns a positive-zero-negative value
 - Result of comparison
 - Depending on actual data we can use relational operators, string comparisons, ...

- Binary Search ovvero ricerca binaria
- Ricerca all'interno di array ordinato
- Stessi parametri della qsort() + valore da ricercare
- Restituisce
 - Indirizzo elemento trovato
 - NULL se ricerca infruttuosa

```
void *bsearch(const void *key, const void *base,  
             size_t nmemb, size_t size,  
             int (*compar)(const void *, const void *));
```



UNIVERSITÀ DI PARMA

Function Pointers



KEEP
CALM
IT'S
QUESTION
TIME

*Memoria est thesaurus omnium rerum et
custos*

De Oratore, Cicerone, I sec a.C.